

Lab 05 Step-by-step Guide to ArrayList, Inheritance, Singleton

Q1 (15 min) => Q2 (25 min)

=> Q3 (10 min)

=> Q4 (10 min) => Q5 (10 min) => Q6,7(? min)

Attention: You will learn a lot! Worth redo!

Q0. Introduction to *static field*

A static field in a class is a single field owned by the whole class
i.e., Not a field per object instance.

What is the output from the following program? What is the difference between static and non-static field? Give your answer on Canvas:

```
class X
{
    private static int a;
    private int b;

    public X(int n)
    {
        this.a = n;
        this.b = n;
    }

    public void print()
    {
        System.out.printf("a is %d, b is %d\n", a, b);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        X x1 = new X(1);
        X x2 = new X(2);
        X x3 = new X(3);
        x1.print();
        x2.print();
        x3.print();
    }
}
```

Q1. Introduction to ArrayList

java.util.ArrayList is a very useful class which is:

- Similar to an array, for storing a collection of object elements
- Automatically adjusts its capacity
- Need a type parameter to specify the type of elements.
Syntax: `ArrayList<Element_type>`
- Add / retrieve / remove elements: `.add(obj)`, `.get(index)`, `.remove(index)`
- The count of elements: `.size()`

Example: An array list of Employee objects

1. Define an array list variable: `ArrayList<Employee> allEmployees;`
2. Create the array list: `allEmployees = new ArrayList<Employee>();`
3. Add employee objects:
`Employee e;`
`e = new Employee(..);`
`allEmployees.add(e);`
4. To get an employee object:
`Employee e;`
`e = allEmployees.get(idx);`

Below is the implementation of the Employee class:

(Note the static method for file reading: `createEmployeeListFromFile`)

```
public class Employee
{
    private String id;
    private String name;
    private double salary;

    .. //Constructor, assessor methods, etc..

    //Get file data into the array list
    public static ArrayList<Employee> createEmployeeListFromFile(String filepathname)
    {
        ArrayList<Employee> arraylist_Staff;
        arraylist_Staff = new ArrayList<Employee>();

        ..
        return arraylist_Staff;
    }
}
```

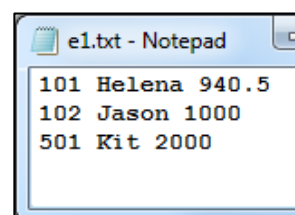
Your task: Complete a program which gets file records of employees into an array list, then lists them.

Given program framework (Main.java and Employee.java) and data file are available for download.

Sample rundown:

```
Please enter the filename: c:\e1.txt

Total count: 3 records.
[101 Helena] Salary: 940.50
[102 Jason] Salary: 1000.00
[501 Kit] Salary: 2000.00
```



Note: You do NOT need to submit Q1 to PASS.

Q2. Introduction to Inheritance

Suppose we have Managers who are

- just like employees
- however, managers get bonuses, **ids of managers start with '9'**

We observe an "is-a" relationship: Every manager **is an** employee.

We model such kind of relationship as a form of *inheritance*. Java keyword - **extends**:

```
class Manager extends Employee
{
    .. added field, eg. bonus
    .. added methods, constructors,
    .. redefine methods in the Employee class,
       e.g. toStringSalaryDetails, getSalary
}
```

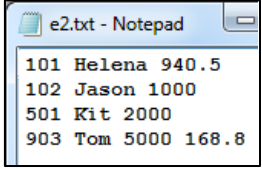
Note:

- We say that Manager is a **subclass**, Employee is the **superclass**
- In the Manager class, we type **super** to refer to methods and fields from the Employee class.
- We can use the array list of employees to hold both employee and manager objects.

Your task:

New set of records and sample rundown:

```
Please enter the filename: c:\e2.txt
Total count: 4 records.
[101 Helena] Salary: 940.50
[102 Jason] Salary: 1000.00
[501 Kit] Salary: 2000.00
[903 Tom] Basic salary: 5000.00, Bonus: 168.80
```



```
101 Helena 940.5
102 Jason 1000
501 Kit 2000
903 Tom 5000 168.8
```

1. Add the Manager class (Refer to given code)
2. Add a PersonnelOffice class which encapsulates the array list of employee/manager records, deals with file reading and reporting. (Refer to given code)

Note that:

- `.createEmployeeListFromFile()` should now be removed from Employee.java
Reason: Employee.java should not handle specific record types like Managers. From now on PersonnelOffice will handle the file reading task.
- The `main()` method now simply creates a PersonnelOffice object and uses it to process all employee records.

3. Rewrite `main()`:

```
public static void main(String[] args) throws FileNotFoundException
{
    Scanner in = new Scanner(System.in);
    System.out.print("Please enter the filename: ");
    String filepathname = in.next();

    PersonnelOffice po = new PersonnelOffice();
    po.loadEmployeeData(filepathname);

    System.out.println("\nTotal count: " + po.getTotal() + " records.\n");
    po.report();

    in.close();
}
```

Submit the finished program (All *.java) to PASS.

Q3. Modify `report()` in `PersonnelOffice.java` to show the total salary expense, like this:

```
Please enter the filename: c:\e2.txt

Total count: 4 records.

[101 Helena] Salary: 940.50
[102 Jason] Salary: 1000.00
[501 Kit] Salary: 2000.00
[903 Tom] Basic salary: 5000.00, Bonus: 168.80
=====
Total salary expense: 9109.30
```

Note: Use `printf` with `%.2f` to **print 2 digits** after the decimal point. [Ref: `Employee's toStringSalaryDetails()`]

Q4. Modify the program to add a percentage to the basic salaries of all employees/managers, like this:

```
Please enter the filename: c:\e2.txt

Total count: 4 records.

[101 Helena] Salary: 940.50
[102 Jason] Salary: 1000.00
[501 Kit] Salary: 2000.00
[903 Tom] Basic salary: 5000.00, Bonus: 168.80
=====
Total salary expense: 9109.30

Enter percentage for raising salary: 0.5

[101 Helena] Salary: 945.20
[102 Jason] Salary: 1005.00
[501 Kit] Salary: 2010.00
[903 Tom] Basic salary: 5025.00, Bonus: 168.80
=====
Total salary expense: 9154.00
```

Your tasks:

- Add a method, `raiseAllSalaries(double percentage)`, to `PersonnelOffice.java`. Then `main()` will call it to add salaries.
(Note that a method called `raiseSalary` is already given in `Employee.java`. You should find it useful.)
- Modify `main()` accordingly.

Q5. Apply the Singleton Pattern

[Singleton: Ensure that only one instance of a class is created and Provide a global access point to the object. -- www.oodeesign.com]

Now we will keep a universal PersonnelOffice object by a static field in the PersonnelOffice class.

Explanation:

Up to Q4, we create an instance of the PersonnelOffice object in main(). However, consider that there should be only one PersonnelOffice object in the whole enterprise, we will change it as:

- Create the universal PersonnelOffice object inside PersonnelOffice.java
We use a *static field* to refer to the object.
(A static field is a single field owned by the whole class, i.e., Not a field per object instance.)
- Provide public access to the universal PersonnelOffice object
- Make the constructor private so that outsiders cannot create another instance of PersonnelOffice.

Given code:

```
public class PersonnelOffice
{
    //Reference to an array list of employee objects
    private ArrayList<Employee> allEmployees;

    //There should be only one personnel office. So we declare it as a static field in the class.
    private static PersonnelOffice thePO = new PersonnelOffice();

    //Return the personnel office
    public static PersonnelOffice getInstance() {return thePO;}

    //Constructor
    private PersonnelOffice()
    {
        allEmployees = new ArrayList<Employee>();
    }
    ..
}
```

```
public static void main(String[] args) ...
{
    ..
    PersonnelOffice po = PersonnelOffice.getInstance();
    ..
}
```

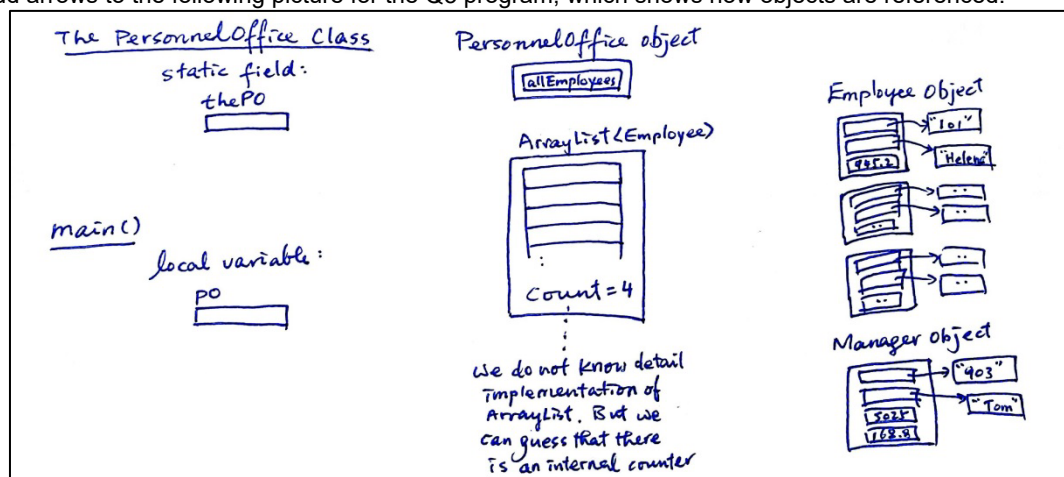
Submit the finished program (All *.java) to PASS.

Lab05 - Progress

(I) Complete the following blanks:

- I have learnt the basic use of **ArrayList** from _____ (Q1 / Q2-4 / Q5)
- I have learnt some basics of **Inheritance** from _____ (Q1 / Q2-4 / Q5)
- I have learnt the idea of **Singleton** from _____ (Q1 / Q2-4 / Q5)

(II) Add arrows to the following picture for the Q5 program, which shows how objects are referenced:

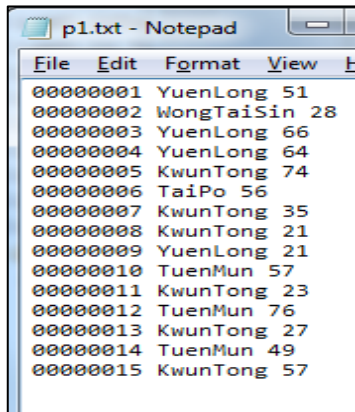


Q6. You are given a program which counts the data in a list of survey records, given in an input file.

An example of the Input file

Each line is a survey record:

- **ID** of the survey record,
- the **area** of survey location,
- the **age** of the interviewee.



Sample Rundown

(Underlined contents are input by user)

Please enter the filename: c:\p1.txt

Statistics:

=====

[Total] Count = 15

[YuenLong area] Count = 4

[KwunTong area] Count = 6

Explanation:

The program now hardcodes to count

- the total number of records
- YuenLong and KwunTong areas

(a) Your task:

(i) Try the program. Study `main()` and the classes: `StatisticsSystem`, `Counter`, `AreaCounter`, `Record`

Represents the `StatisticsSystem` as a Singleton

It keeps an `ArrayList` of counters

```

public static void main(..)
{
    //Read input file pathname
    Scanner in = new Scanner(System.in);
    System.out.print("Please enter the filename: ");
    String filepathname = in.next();

    //Grab the StatisticsSystem ss and add counters
    StatisticsSystem ss = StatisticsSystem.getInstance();
    ss.addCounter(new Counter());
    ss.addCounter(new AreaCounter("YuenLong"));
    ss.addCounter(new AreaCounter("KwunTong"));

    //The ss will load file data and tell its counters to count
    ss.countData(filepathname);

    //The ss will tell its counters to report
    ss.report();

    in.close();
}
  
```

The `Record` class is just for survey data (Reading file and checking data)

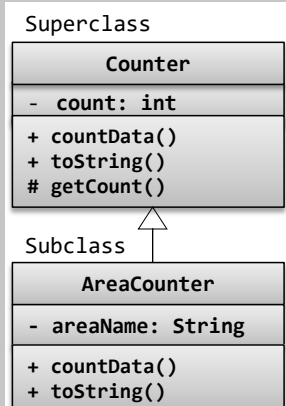
You can find the creation and use of `Record` objects in the `countData` method inside `StatisticsSystem.java`.

You do not need to draw objects of the `Record` class in Q7.

Two types of counters:

- `Counter` (for all records)
- `AreaCounter` (for a specific area)

Note the use of inheritance:



(ii) Rewrite the program so that the user can choose the areas to count:

Note: From **given_code.txt**, learn how to use `.next()` to read from a string (not keyboard!! not file!!)

Sample Rundown

(Underlined contents are input by user)

```
Please enter the filename: c:\p1.txt

Enter the area names (e.g. TaiPo YuenLong WongTaiSin KwunTong): YuenLong WongTaiSin KwunTong

Statistics:
=====
[Total] Count = 15
[YuenLong area] Count = 4
[WongTaiSin area] Count = 1
[KwunTong area] Count = 6
```

(b) Add a new type of counters: AgeGroupCounter, which count age groups.

Note: AgeGroupCounter should also be a subclass of Counter.

The new run-down should be like this:

Sample Rundown

(Underlined contents are input by user)

```
Please enter the filename: c:\p1.txt

Enter the area names (e.g. TaiPo YuenLong WongTaiSin KwunTong): TaiPo YuenLong

Enter the age groups ('-1 -1' to end): 0 50
Enter the age groups ('-1 -1' to end): 51 100
Enter the age groups ('-1 -1' to end): 30 55
Enter the age groups ('-1 -1' to end): -1 -1

Statistics:
=====
[Total] Count = 15
[TaiPo area] Count = 1
[YuenLong area] Count = 4
[Age 0 to 50] Count = 7
[Age 51 to 100] Count = 8
[Age 30 to 55] Count = 3
```

Q: How to **get input until -1, -1**?

A: See the hint at the course web.

Submit the finished program (All *.java) to PASS.

   **Note: Re-do on your own, don't look at the hints/guidelines**   

Q7. Illustrate all key fields, variables, objects, and reference links for the above rundown of Q6(b). They should be drawn like the picture you completed on page 5 for Q5.

Submit you drawing on Canvas

You do not need to draw objects of the Record class.